EUI · SPRING 2026 BIG DATA ENGINEERING PROJECT #2

E-Commerce Behavioral Analytics & Recommendation Engine

// end-to-end pipeline: raw logs → NoSQL profiles → targeted campaign

⚡ APACHE SPARK | 🗄 MONGODB / REDIS | 📦 10 GB+ DATASET | 3 PHASES · 4 TASKS

Step into the role of a **Data Engineer at a high-growth e-commerce startup**. The company has accumulated a massive archive of raw, unstructured web-server logs detailing user interactions — clicks, views, cart additions, and purchases. Your objective is to build an **interconnected, end-to-end data pipeline** across three phases: distributed processing with Spark, NoSQL storage for real-time querying, and an enrichment layer that powers a targeted marketing campaign.

⚠️ The raw dataset is **significantly larger than available RAM**. Standard iterative loops or in-memory libraries like Pandas will crash with an OOM error. You **must** use Apache Spark and strictly apply the **MapReduce paradigm** to distribute computation across CPU cores.

TECH STACK

⚡
PROCESSING
Apache Spark

🗄
STORAGE
NoSQL Database

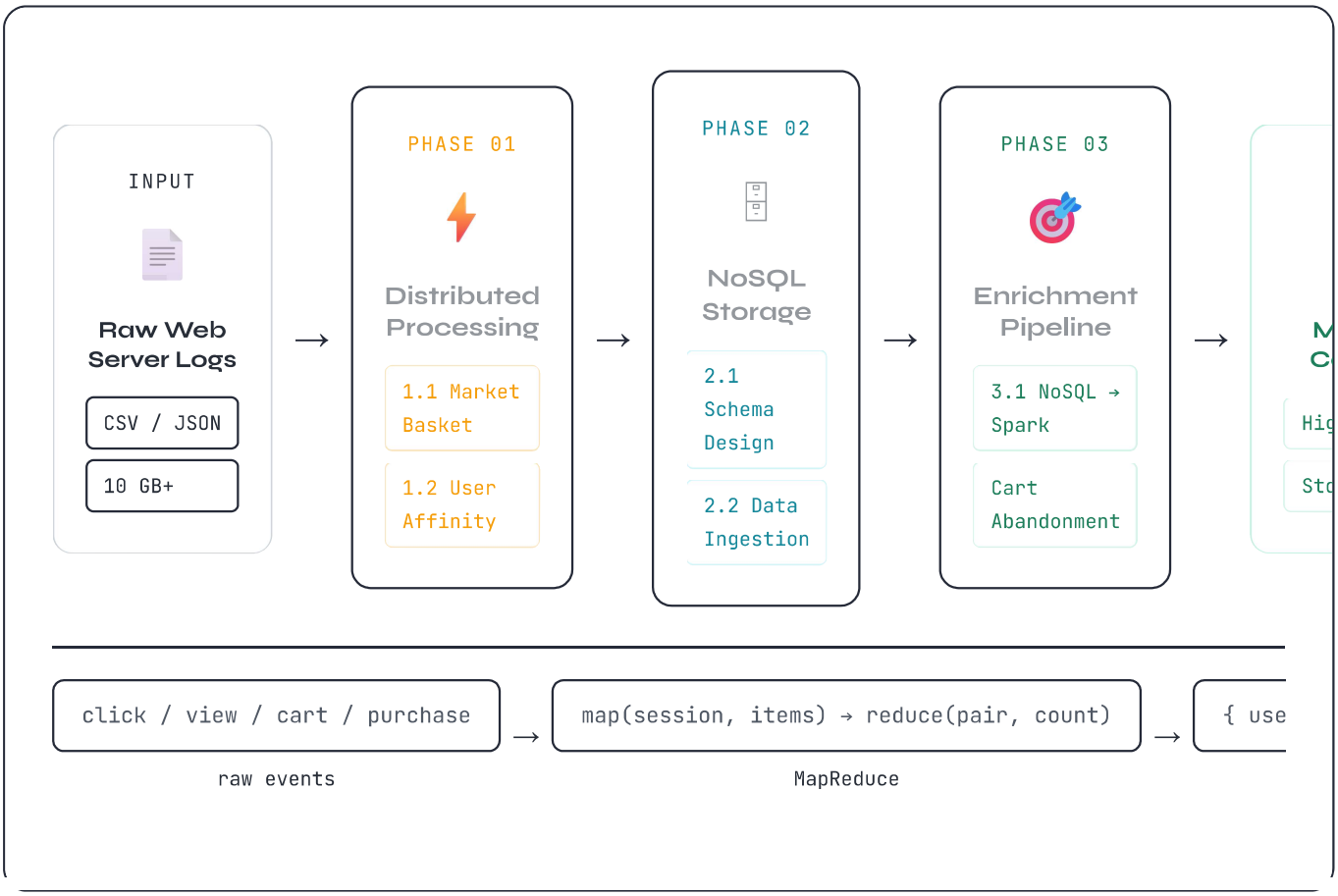
📂
DATASET
10 GB+ Raw Logs

PySpark · Scala · local[*]

MongoDB or Redis · Docker

CSV / JSON · eCommerce events

PROJECT PIPELINE



PHASE DETAILS

PHASE 01 Distributed Data Processing — The MapReduce Paradigm

TASK 1.1

Market Basket Analysis — Item Co-occurrence

OBJECTIVE Determine which items are most frequently purchased together in the same session across the entire dataset.

REQUIREMENT Formulate a **MapReduce strategy** that groups items from the same session and reduces them to find the global frequency of co-purchased pairs.

MAP STEP Emit `(session_id, item_id)` pairs for every purchase event in the log.

REDUCE STEP	Group by session, generate all item pairs, count global frequency of each pair.

TASK 1.2

User Affinity Aggregation

OBJECTIVE	Build a behavioral profile for each user based on their historical actions.
REQUIREMENT	Apply MapReduce to assign **weighted scores** to different user events and aggregate per user.
WEIGHTS	`View = 1` `Cart = 3` `Purchase = 5`
OUTPUT	Each user's top-affinity product categories ranked by weighted score sum.

PHASE 02 NoSQL Data Modeling & Storage

TASKS 2.1 & 2.2

Schema Design & Data Ingestion

OBJECTIVE	Design a data model optimized for **instant lookups** in real-time web recommendation queries, then load Phase 1 output into it.
CONSTRAINT	Relational databases are unsuited for the read-heavy, low-latency requirements of the web frontend. A NoSQL approach is mandatory.

OPTION A

MongoDB

Design a **denormalized document schema.** Embed top categories and co-occurrence data directly into user documents for single-query retrieval.

OPTION B

Redis

Define a **Key-Value naming convention** and leverage appropriate data structures — Strings, Hashes, or Sorted Sets for ranked affinities.

PHASE 03 Data Enrichment & The Interconnected Pipeline

TASK 3.1

NoSQL-to-Spark Integration — Cart Abandonment Recovery

GOAL

The marketing team wants to send **high-value discount codes** only when the abandoned item belongs to a user's top affinity categories — saving budget by targeting intelligently.

01 Read a new batch of raw logs using Spark and **filter for users** who added an item to their cart but did *not* trigger a `purchase` event in that session.

02 Configure Spark to **query historical User Profiles** directly from your chosen NoSQL database (MongoDB connector or Redis client).

03 **Join the two datasets** using Spark transformations. If the abandoned item's category matches one of the user's top categories → flag `High_Discount`. Otherwise → flag `Standard_Reminder`.

DELIVERABLES

01

Source Code

Spark scripts for Phase 1 and Phase 3, clearly demonstrating the Map and Reduce transformations used to achieve each objective.

`.py · .scala`

02

Architecture & Schema Document

Diagram or JSON/Key-Space documentation of your NoSQL schema, plus a written justification of your database choice over a SQL table.

`.pdf`

03

Query Demo Script

A standalone script that connects to your local NoSQL database and fetches recommendations for a given `user_id` and `item_id` — proving instant lookup capability.

`.py · .js · any`